Graphs

assadullahkhan

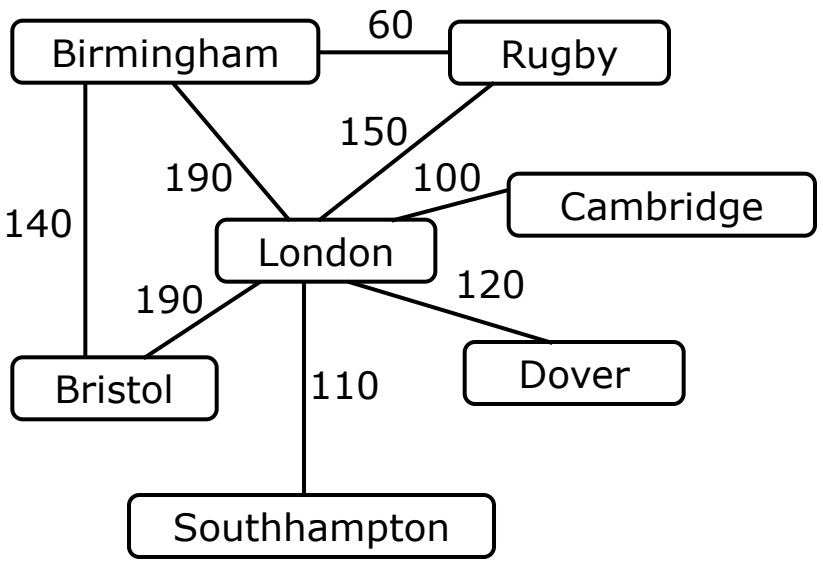
AssadUllahKhan

Graph terminology II

- The size of a graph is the number of *nodes* in it
- The empty graph has size zero (no nodes)
- If two nodes are connected by an edge, they are neighbors (and the nodes are adjacent to each other)
- The degree of a node is the number of edges it has
- For directed graphs,
  - If a directed edge goes from node S to node D, we call S the source and D the destination of the edge
    - The edge is an out-edge of S and an in-edge of D
    - S is a predecessor of D, and D is a successor of S
  - The in-degree of a node is the number of in-edges it has
  - The out-degree of a node is the number of out-edges it has

Graph terminology III

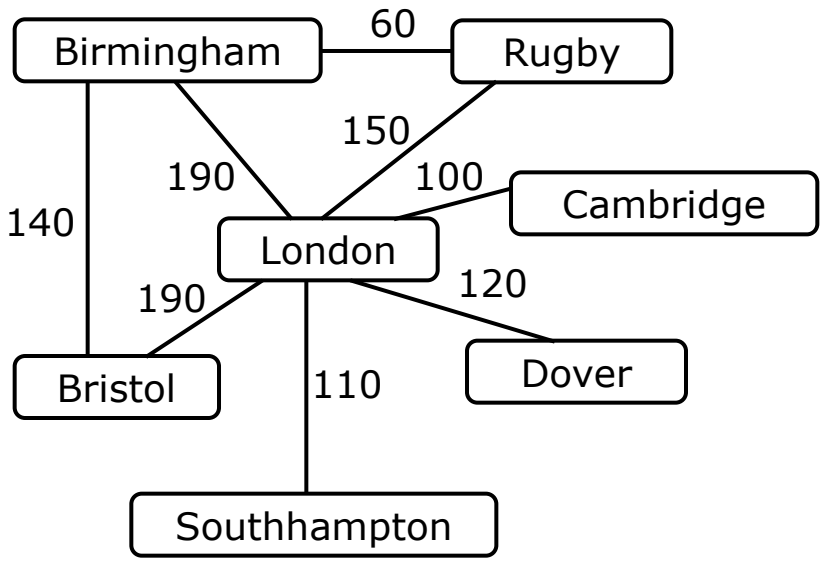
- A path is a list of edges such that each node (but the last) is the predecessor of the next node in the list
- A cycle is a path whose first and last nodes are the same



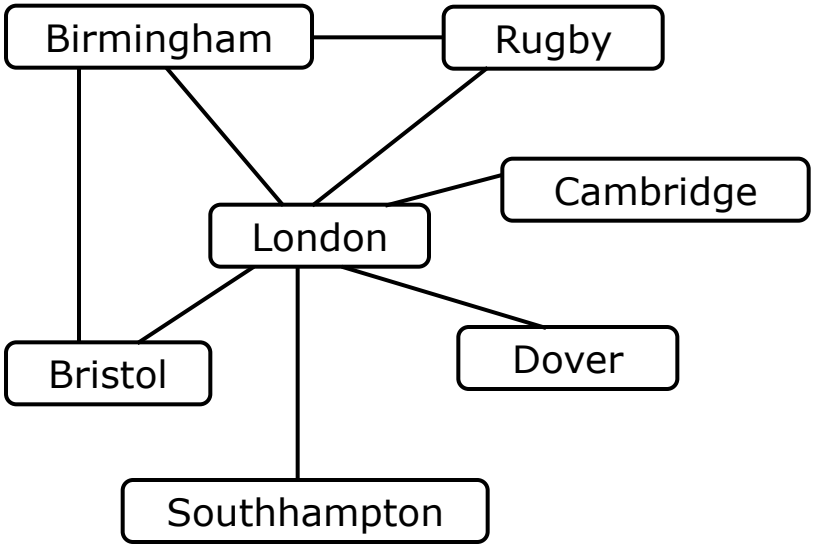
- Example: (London, Bristol, Birmingham, London, Dover) is a path
- Example: (London, Bristol, Birmingham, London) is a cycle
- A cyclic graph contains at least one cycle
- An acyclic graph does not contain any cycles

Definitions: Weighted Graph

- A weighted graph is a triple (V, E, W)
  - where (V, E) is a graph (directed or undirected) and
  - W is a function from E into R, the reals (integer or rationals).
  - For an edge E, W(E) is called the weight of E or Cost of E.
  - Cost of London to Dover is 120



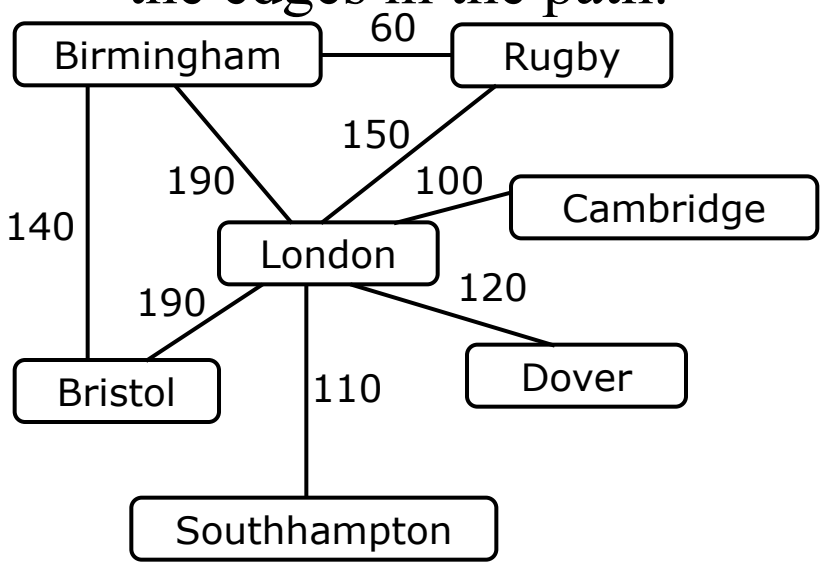
Weighted Graph



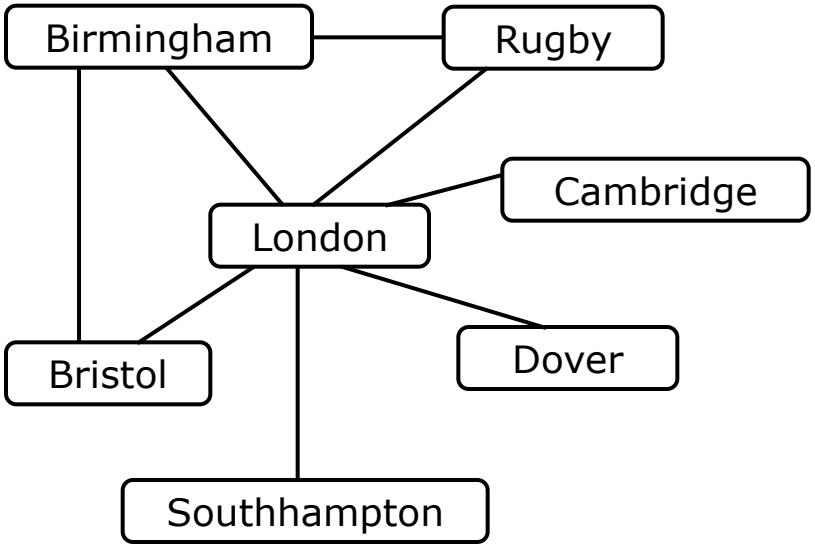
Un-weighted Graph

Continue…

- **Length of Path:** No. of edges connecting vertices in the sequence of the vertices in the path.
  - It is equal to the number of vertices in the path minus one.
- **Cost of path:** For un-weighted graph path length is the number of edges in the path, specifically n−1.
  - For weighted graph path length is the sum of the weights of all the edges in the path.



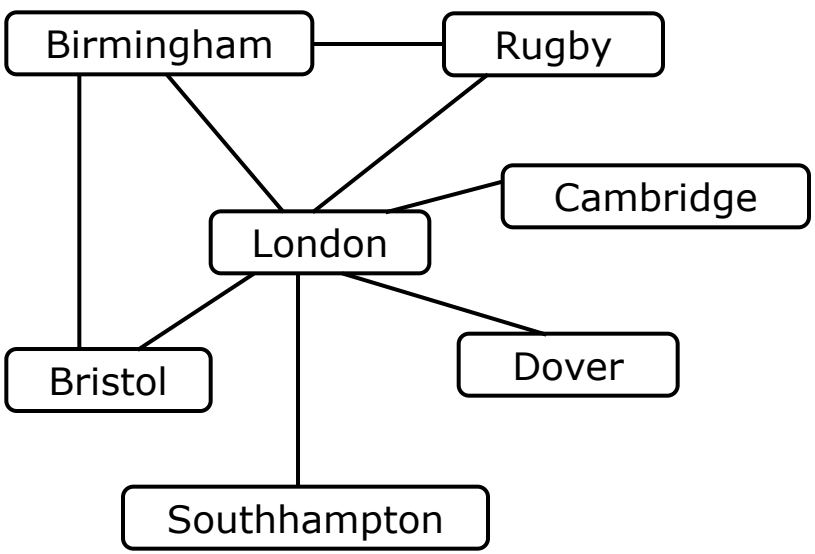
Weighted Graph



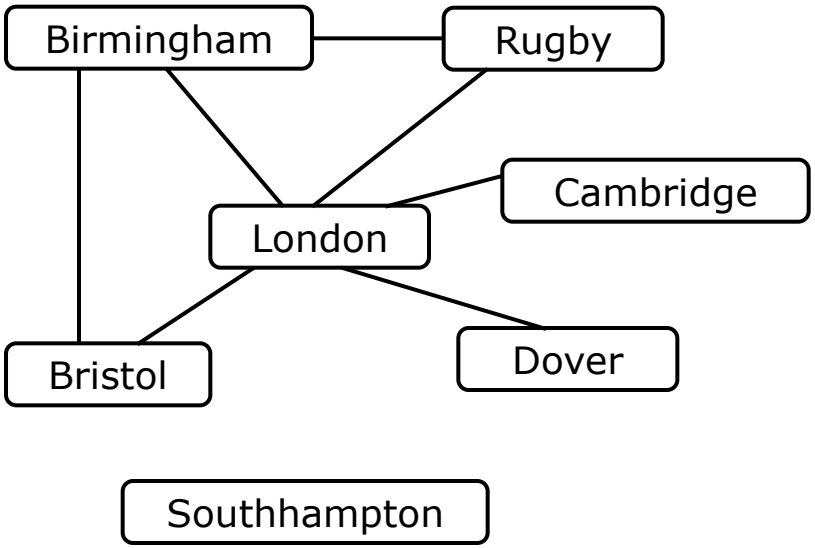
Un-weighted Graph

Definitions: Connected Graph

- A Graph G is said to be connected if there is a path between any of its nodes.
  - In the following figure (a) every node is connected to all other nodes therefore it is connected graph.
  - In figure(b) Southampton is disconnected from all other node



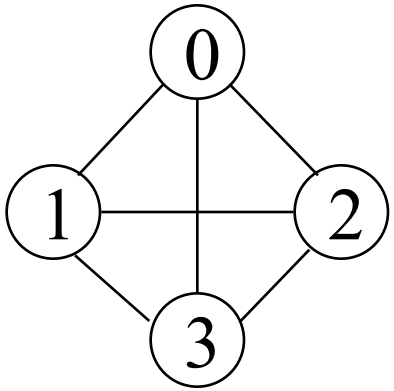
(a) Connected Graph



(b) Disjoint Graph

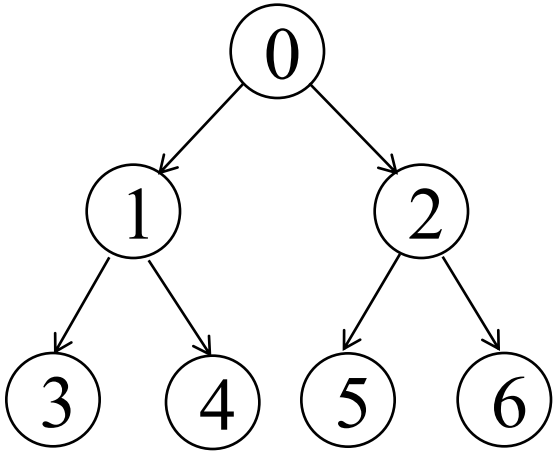
Definitions: Different Types of Graph

connected

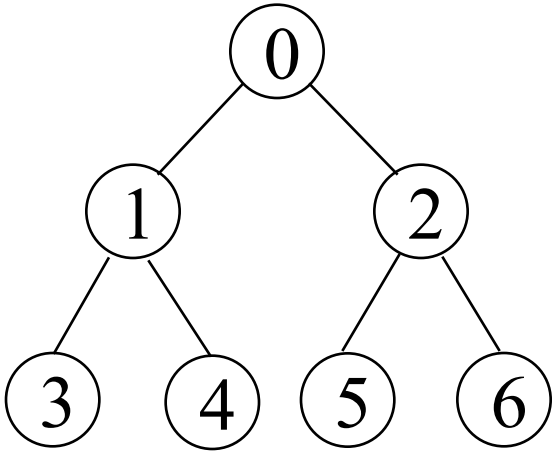


G_1

Cyclic Graph
Undirected



Tree (acyclic graph)
Directed



G_2

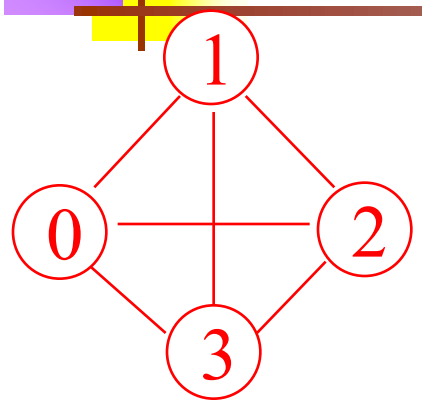
Tree (acyclic graph)
Undirected

*Note: A **tree** is a special kind of graph that is connected and acyclic*

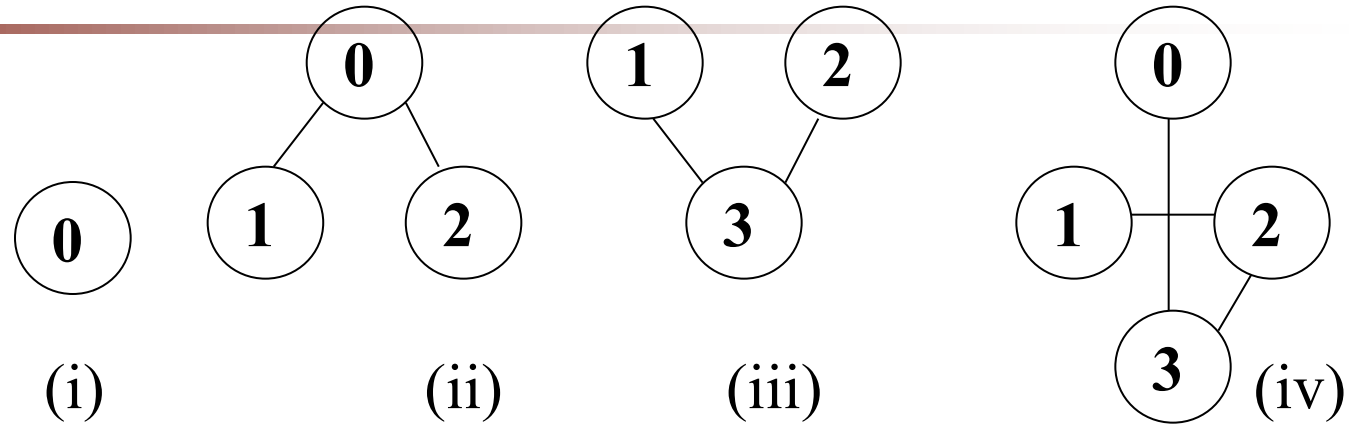
Subgraph and Path

- A subgraph of G is a graph G' such that V(G') is a subset of V(G) and E(G') is a subset of E(G)
- A path from vertex v_p to vertex v_q in a graph G, is a sequence of vertices, v_p, v_{i1}, v_{i2}, ..., v_{in}, v_q, such that (v_p, v_{i1}), (v_{i1}, v_{i2}), ..., (v_{in}, v_q) are edges in an undirected graph
- The length of a path is the number of edges on it

Subgraph and Path



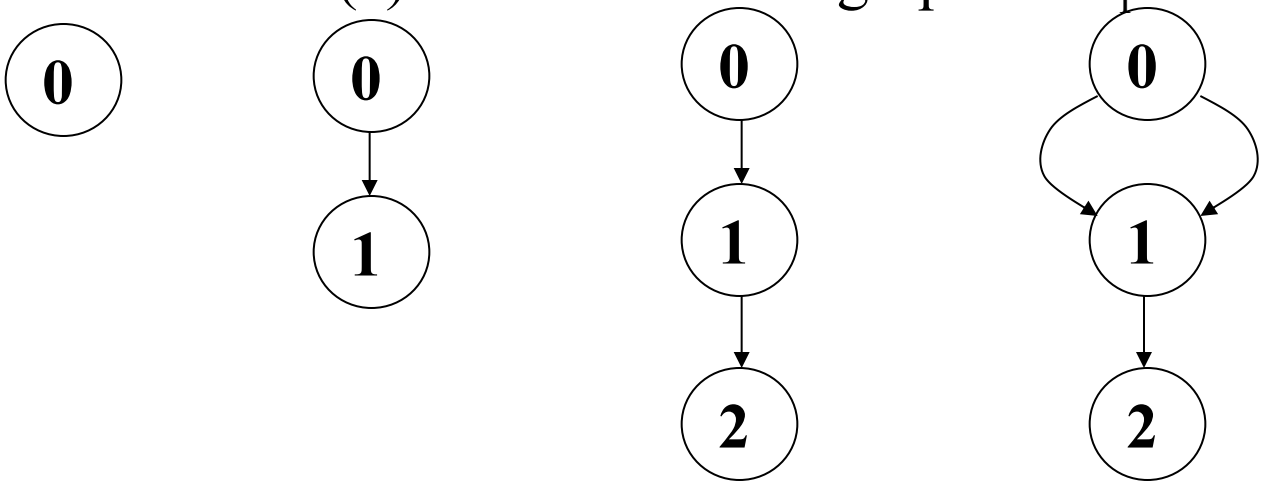
G_1



(i) (ii) (iii) (iv)

(a) Some of the subgraph of G_1

G_3

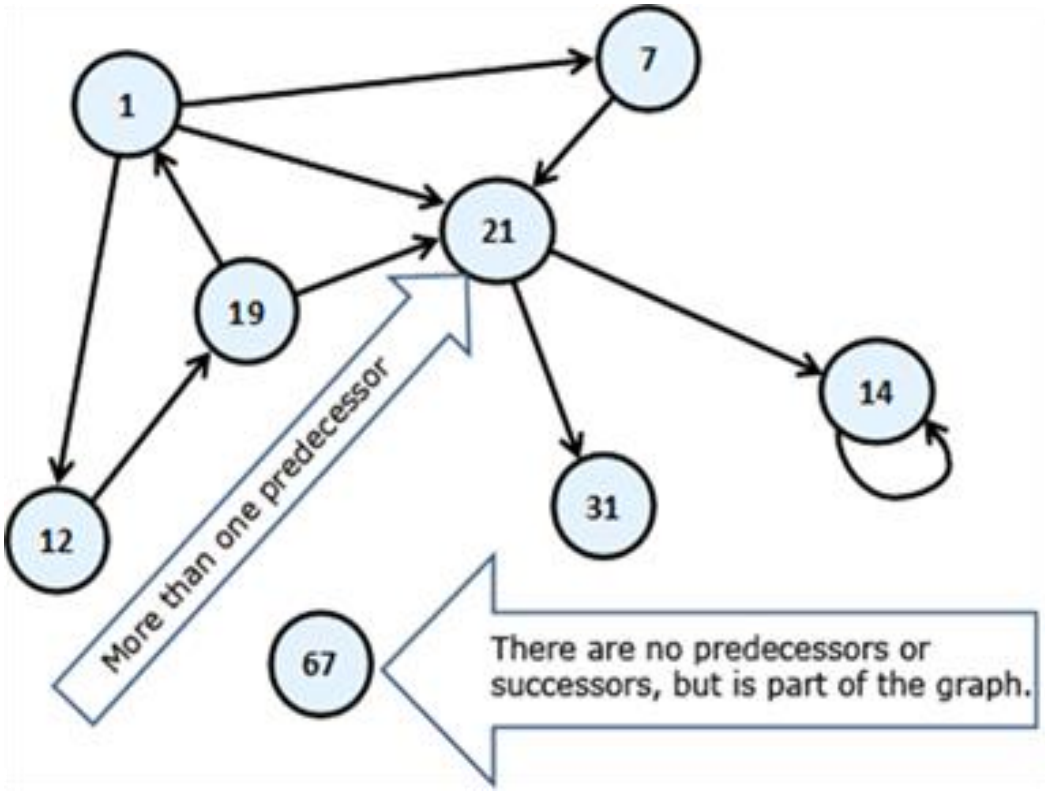


(i) (ii) (iii) (iv)

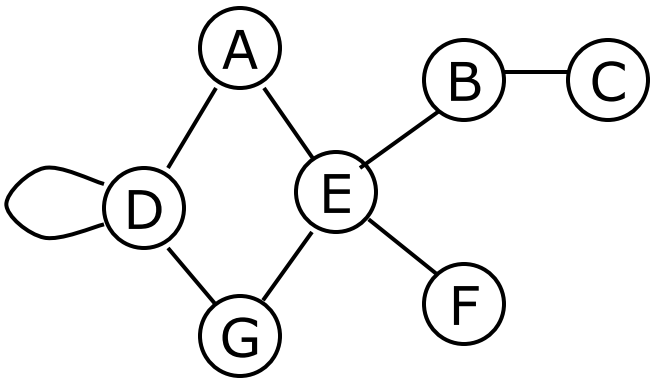
(b) Some of the subgraph of G_3

Identify?

1. V={ }
2. E={ }
3. Directed or Undirected
4. Cyclic or Acyclic
5. Complete or incomplete
6. Directed Acyclic or Cyclic
7. Degree of Node 19
8. Indegree of Node 21
9. Outdegree of Node 21
10. Path from 19 to 21
11. Path from 7 to 12
12. Subgraph of 1, 19, 12
13. Identify Cycles?
14. Adjacent Nodes of 19
15. Size of the Graph
16. Length of Path from 7 to 31
17. Connected or Disjoint
18. Weighted or Unweighted



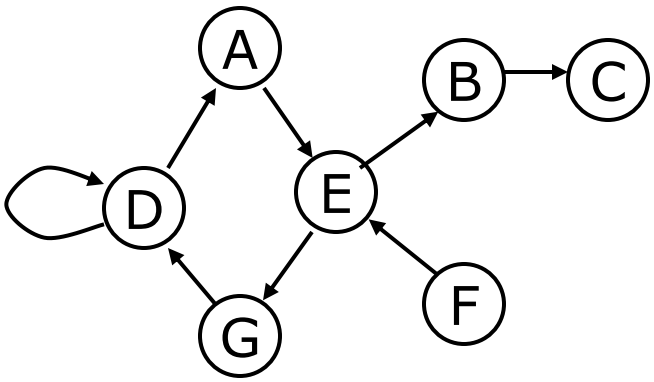
Adjacency-matrix representation I



	A	B	C	D	E	F	G
A				●	●		
B			●		●		
C		●					
D	●			●			●
E	●	●				●	●
F					●		
G				●	●		

- One simple way of representing a graph is the adjacency matrix
- A 2-D array has a mark at [i][j] if there is an edge from node i to node j
- The adjacency matrix is symmetric about the main diagonal
- This representation is only suitable for *small* graphs! (Why?)

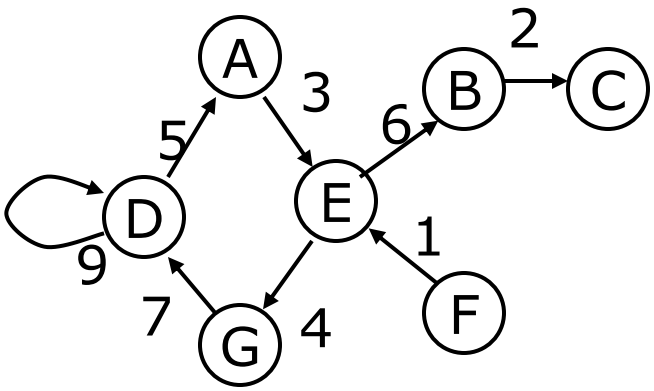
Adjacency-matrix representation II



	A	B	C	D	E	F	G
A					●		
B			●				
C							
D	●			●			
E		●					●
F					●		
G				●			

- An adjacency matrix can equally well be used for digraphs (directed graphs)
- A 2-D array has a mark at `[i][j]` if there is an edge from node `i` to node `j`
- Again, this is only suitable for *small* graphs!

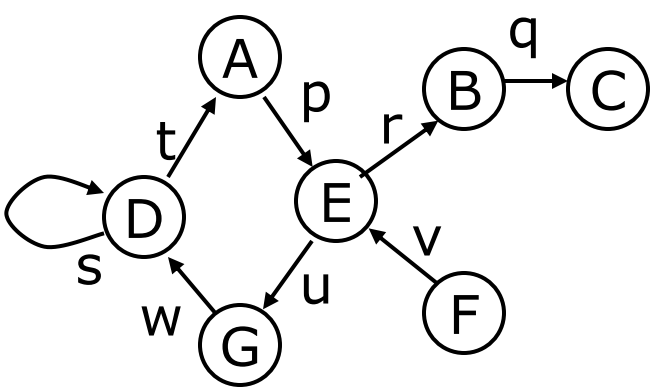
Adjacency-matrix representation II



	A	B	C	D	E	F	G
A					3		
B			2				
C							
D	5			9			
E		6					4
F					1		
G				4			

- An adjacency matrix for weighted graph is constructed using weights of edges for digraphs (directed graphs)
- A 2-D array has a mark at [i][j] if there is an edge from node i to node j
- In the resultant matrix we will only show the edge values of the graph

Edge-set representation II



nodeSet = {A, B, C, D, E, F, G}

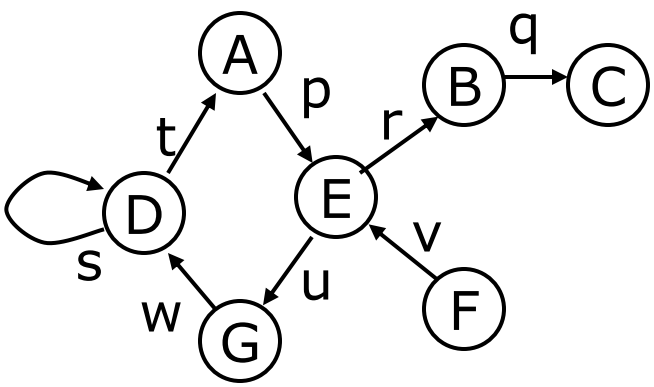
edgeSet = { p: (A, E),
q: (B, C), r: (E, B),
s: (D, D), t: (D, A),
u: (E, G), v: (F, E),
w: (G, D) }

- Here we have a set of nodes, and each node contains only its element (not shown)
- Each edge contains references to its source and its destination (and its attribute, if any)

Edge-set representation I

- An edge-set representation uses a *set* of nodes and a *set* of edges
  - The sets might be represented by, say, linked lists
  - The set links are stored in the nodes and edges themselves
- The only other information in a node is its element (that is, its value)—it does not hold information about its edges
- The only other information in an edge is its source and destination (and attribute, cost, if any)
  - If the graph is undirected, we keep links to both nodes, but don't distinguish between source and destination
- This representation makes it easy to find nodes from an edge, but you must search to find an edge from a node
- This is seldom a good representation

Edge-set representation II



nodeSet = {A, B, C, D, E, F, G}

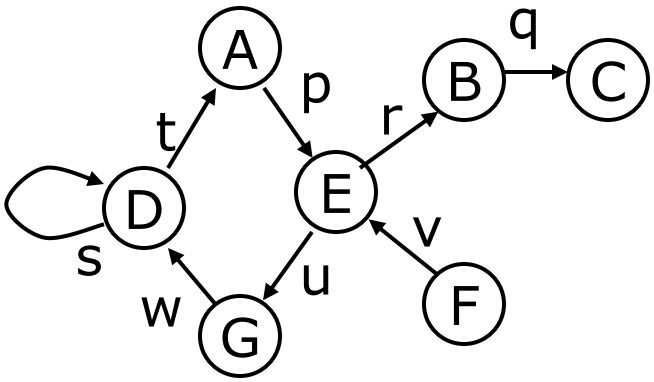
edgeSet = { p: (A, E),
q: (B, C), r: (E, B),
s: (D, D), t: (D, A),
u: (E, G), v: (F, E),
w: (G, D) }

- Here we have a set of nodes, and each node contains only its element (not shown)
- Each edge contains references to its source and its destination (and its attribute, if any)

Adjacency-set representation I

- An adjacency-set representation uses a *set* of nodes
  - Each node contains a reference to the set of *its* edges
  - For a directed graph, a node might only know about (have references to) its out-edges
- Thus, there is not one single edge set, but rather a separate edge set for each node
  - Each edge would contain its attribute (if any) and its destination (and possibly its source)

Adjacency-set representation II



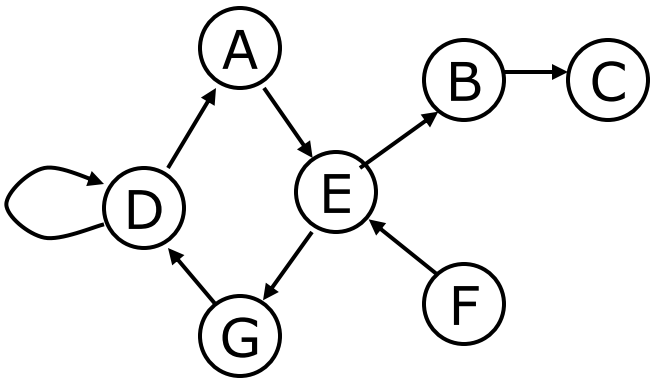
A →	{ p }	p: (A, E)
B →	{ q }	q: (B, C)
C →	{ }	r: (E, B)
D →	{ s, t }	s: (D, D)
E →	{ r, u }	t: (D, A)
F →	{ v }	u: (E, G)
G →	{ w }	v: (F, E)
		w: (G, D)

- Here we have a set of nodes, and each node refers to a set of edges
- Each edge contains references to its source and its destination (and its attribute, if any)

Adjacency-set representation III

- If the edges have no associated attribute, there is no need for a separate `Edge` class
  - Instead, each node can refer to a set of its *neighbors*
  - In this representation, the edges would be implicit in the connections between nodes, not a separate data structure
- For an undirected graph, the node would have references to all the nodes adjacent to it
- For a directed graph, the node might have:
  - references to all the nodes adjacent to it, or
  - references to only those adjacent nodes connected by an out-edge from this node

Adjacency-set representation IV



A	{ E }
B	{ C }
C	{ }
D	{ D, A }
E	{ B, G }
F	{ E }
G	{ D }

- Here we have a set of nodes, and each node refers to a set of other (pointed to) nodes
- The edges are *implicit*

Searching a graph

- With certain modifications, any tree search technique can be applied to a graph
  - This includes depth-first, breadth-first, depth-first iterative deepening, and other types of searches
- The difference is that a graph may have cycles
  - We don't want to search around and around in a cycle
- To avoid getting caught in a cycle, we must keep track of which nodes we have already explored
- There are two basic techniques for this:
  - Keep a set of already explored nodes, or
  - Mark the node itself as having been explored
    - Marking nodes is not always possible (may not be allowed)

Graph applications

- Graphs can be used for:
  - Finding a route to drive from one city to another
  - Finding connecting flights from one city to another
  - Determining least-cost highway connections
  - Designing optimal connections on a computer chip
  - Implementing automata
  - Implementing compilers
  - Doing garbage collection
  - Representing family histories
  - Doing similarity testing (e.g. for a dating service)
  - Pert charts
  - Playing games

Finding connected components

- A depth-first search can be used to find connected components of a graph
  - A connected component is a set of nodes; therefore,
  - A set of connected components is a set of sets of nodes
- To find the connected components of a graph:

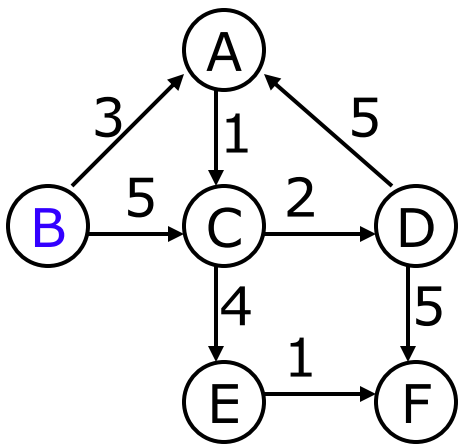
```
while there is a node not assigned to a component {
    put that node in a new component
    do a DFS from the node, and put every node
        reached into the same component
}
```

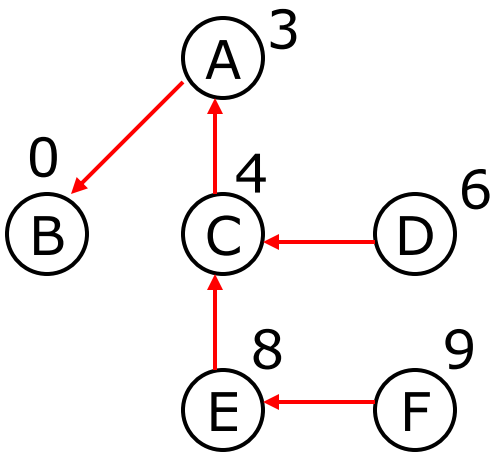
Shortest-path

- Suppose we want to find the shortest path from node X to node Y
- It turns out that, in order to do this, we need to find the shortest path from X to *all* other nodes
  - Why?
  - If we don't know the shortest path from X to Z, we might overlook a shorter path from X to Y that contains Z
- Dijkstra's Algorithm finds the shortest path from a given node to *all* other reachable nodes

Dijkstra's algorithm I

- Dijkstra's algorithm builds up a *tree:* there is a path from each node back to the starting node
- For example, in the following graph, we want to find shortest paths from node B





- Edge values in the graph are weights
- Node values in the tree are *total* weights
- The arrows point in the *right direction* for what we need (why?)

Dijkstra's algorithm II

- For each vertex v, Dijkstra's algorithm keeps track of three pieces of information:
  - A boolean telling whether we *know* the shortest path to that node (initially true only for the starting node)
  - The length of the shortest path to that node known so far (0 for the starting node)
  - The predecessor of that node along the shortest known path (unknown for all nodes)
- Dijkstra's algorithm proceeds in phases—at each step:
  - From the vertices for which we don't know the shortest path, pick a vertex v with the smallest distance known so far
  - Set v's "known" field to true
  - For each vertex w adjacent to v, test whether its distance so far is greater than v's distance plus the distance from v to w; if so, set w's distance to the new distance and w's predecessor to v

Summary

- A graph may be directed or undirected
- The edges (=arcs) may have weights or contain other data, or they may be just connectors
- Similarly, the nodes (=vertices) may or may not contain data
- There are various ways to represent graphs
  - The "best" representation depends on the problem to be solved
  - You need to consider what kind of access needs to be quick or easy
- Many tree algorithms can be modified for graphs
  - Basically, this means some way to recognize cycles

The End